



HACK THE BOX · LINUX

Lame

Complete technical report ·
documented RETRO case

Lame machine solved

HTB Lame · 2025-05

Author	Ramon Santos Sabarich / r4ms4nt
Type	Complete technical report ready for archiving
Objective	Document enumeration, CVE, user and root with clear reusable traceability
Date	2025-05



Standardized technical report for archiving,
reference, and traceability.

READING GUIDE

Contents

Final English technical report prepared for archiving, study, and consultation. The real chain of the case is preserved, verified steps are distinguished clearly, and the point that remains pending verification stays visible.

- Case introduction
- Preparation and case startup
- Initial enumeration
- Identification of the first dominant surface: VSFTPd
- Validation of the VSFTPd line: test, result, and discard
- Samba enumeration and selection of the main path
- Exploitation validated through CVE-2007-2447
- Obtaining user.txt
- Obtaining root.txt
- Complementary technical observations
- Final technical summary of the real chain
- Reusable lessons for PENTEST-STUDIO

Validated technical chain

Brief initial enumeration.

VSFTPd 2.3.4 identified and tested without a useful session.

Samba 3.0.20 associated with CVE-2007-2447 as the main path.

Useful shell obtained as root.

Later reading of user.txt and root.txt from privileged context.

Case introduction

Lame is a retired **Hack The Box** machine reconstructed here as a **RETRO** case within **PENTEST-STUDIO**. The purpose of this document is not to simulate a new resolution or artificially polish the exploitation, but to turn the preserved material into a solid, faithful, and reusable study report.

The technical chain that is validated by the available evidence is simple, but very useful from a training perspective:

- brief initial enumeration;
- detection of two potentially vulnerable lines;
- discard of a famous but non-productive path;
- shift of focus to a more exploitable legacy service;
- direct acquisition of root context;
- reading of user .txt and root .txt already from that privileged context.

This case teaches something important: a simple machine does not have to produce poor documentation. Precisely because the chain is short, it is worth explaining very clearly the reasons behind each decision, what was actually observed, and which part of the case remains pending more precise verification.

Preparation and case startup

Because this is a RETRO case, the preparation phase is not about reconstructing an environment from scratch, but about reading the case as a historical technical record. That changes the documentation approach:

- no new lab session is described;
- no instrumentation that does not appear in the sources is added;
- the exploitation is not rewritten as if it had been performed today with another methodology;
- the work is based on the **actual observed chain**.

For that reason, each phase in this document answers a specific question:

- what was actually observed?
- why did that step make sense?
- what technical reading was made from the result?
- what changed from that point onward?

That approach makes it possible to transform exploitation notes or historical material into a much more useful learning document than a simple command list.

Initial enumeration

What was done

The initial enumeration starts from a scan of the **top 1000 TCP ports** followed by filtering the open ports:

```
nmap -v -T4 -Pn --top-ports 1000 -oA nmap/top1000_tcp 10.129.56.2
grep open nmap/top1000_tcp.nmap
```

Why this step made sense

On a simple or entry-level machine, scanning the most common ports is often enough to identify most of the exploitable surface without immediately moving into a full 65535-port sweep. Here the goal was not to exhaust every possibility from the first minute, but to obtain a **quick reading of the dominant services**.

The command used combines several useful decisions:

- -v increases verbosity to follow scan progress;
- -T4 speeds up the rhythm without becoming extremely aggressive;
- -Pn avoids relying on the initial ping;
- --top-ports 1000 prioritizes the most likely surface;
- -oA stores the output in several formats for later review;
- grep open simplifies the final reading and leaves only relevant services.

What was actually observed

The verified result was:

- 21/tcp
- 22/tcp
- 139/tcp
- 445/tcp

Total number of open ports in this phase: **4**.

How to interpret the finding

This first result already organizes the machine into three main lines:

- **FTP** on 21/tcp
- **SSH** on 22/tcp
- **SMB / NetBIOS** on 139/tcp and 445/tcp

Not all of those lines deserve the same documentary weight.

The preserved evidence truly develops two branches:

- the **FTP** line, because there is a known vulnerable version that had to be checked;

- the **Samba** line, because it is ultimately the path that does lead to valid exploitation.

22/tcp remains visible and should not be erased, but it should not be overemphasized either: in the available material it **does not appear as an exploited path or a useful pivot**.

What changes afterward

Once the surface is visible, the next logical decision is to **identify versions** in the most promising services. Given the well-known history of vsFTPD 2.3.4 and the classic relevance of old Samba in HTB, the case naturally moves toward those two branches.

Reusable lesson

Initial enumeration is not just about "seeing ports". Its real value lies in **prioritizing investigation lines**. In this case, that avoids two common mistakes:

- wasting time on a visible service that is not supported by the case evidence;
- jumping too quickly to exploitation without first confirming the real service version.

Identification of the first dominant surface: VSFTPD

Version detection

To confirm the identity of the FTP service, the following was executed:

```
nmap -sV -p21 -oA nmap/ftp_version 10.129.56.2
```

What this command is looking for

At this point, it is no longer important to know whether the port is open—that was already validated in the previous phase—but rather **which exact version** of the service is responding on 21/tcp.

Here -sV is the key piece, because it allows the workflow to move from a generic intuition ("there is FTP") to a much more precise technical hypothesis ("there is a specific potentially vulnerable version"). The scan is limited to port 21 because the objective is already surgical: confirm the FTP service version and decide whether it deserves an exploitation test.

What was actually observed

The detected service was:

- vsFTPD 2.3.4

Why this finding matters

vsFTPD 2.3.4 is a very well-known version because of its historical association with a backdoor. When it appears in a lab machine, ignoring it would not be reasonable. But here it is worth remembering an important methodological rule:

An apparently vulnerable version does not automatically equal a valid resolution path.

Detecting the version justifies testing that line, but it still does not validate it as the main chain of the case.

Validation of the VSFTPD line: test, result, and discard

What was executed

The preserved sequence for testing the VSFTPD path was:

```
msfconsole
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 10.129.56.2
run
```

Why it made sense to use this path at this point

Once version 2.3.4 had been identified, the reasonable thing was to check whether the associated backdoor was still exploitable on the machine. This test does not appear as a rushed jump into Metasploit, but as **hypothesis validation**:

1. port 21/tcp is open;
2. the service is vsFTPD 2.3.4;
3. there is a historical exploit associated with it;
4. therefore, the line deserves a direct check.

What the sequence does exactly

- `msfconsole` opens the exploitation environment.
- `use exploit/unix/ftp/vsftpd_234_backdoor` loads the specific module for that vulnerability.
- `set RHOSTS 10.129.56.2` defines the target.
- `run` launches the test.

What was actually observed

The historical material validates this:

- the exploit **runs**;
- but it **does not return a useful session**.

How this result should be read

This is one of the most didactic points in the case. The line should not be described as "failed" in an absolute sense, nor as "the machine was meant to be solved this way but something went wrong". The prudent reading is more precise:

- the line **was tested**;
- the test **did not produce the documented useful access**;
- therefore, the branch **cannot be treated as the main resolution path**.

This matters because many weak narratives turn a famous version into the central story of the case even when the evidence says otherwise. That should not be done here.

What changed afterward

The practical discard of the FTP path forces the workflow to return to the remaining surface with a clear mindset: a stronger second line must be found. That second line will be **Samba**.

Reusable lesson

A known path is not discarded just because it "looks old", nor kept just because "it is famous". It is tested, the result is observed, and a decision is made based on evidence. That discipline is much more valuable than memorizing exploit names.

Samba enumeration and selection of the main path

What was executed

Samba identification was supported by this command:

```
nmap -sV -Pn -p139,445 --script=smb-protocols,smb-os-discovery,smb2-security
```

Why this command makes sense here

After confirming that FTP does not produce a useful session, the next strong service on the surface is SMB on 139/445. At this point, launching a blind test is not enough: it is necessary to **confirm the version** and obtain additional context.

This command does not only ask "is there Samba?", but tries to obtain a richer image of the service:

- -sV seeks the exact version;
- -Pn keeps the same operational criterion used in the case;
- -p139,445 focuses on SMB / NetBIOS ports;
- the SMB scripts add protocol and discovery context;
- -oA preserves the output for traceability.

What was actually observed

The detected version was:

- Samba 3.0.20

The historical material associates that version with:

- CVE-2007-2447
- relevant condition: use of `username map script` in `smb.conf`

What this finding means

This is where the machine stops being a simple sequence of tests and begins to have a **clear dominant line**.

The situation is:

- FTP has already been validated as **tested but non-productive**;
- Samba appears with an old version and a very well-known historical vulnerability;
- the source not only mentions the CVE, but also the relevant technical condition;
- therefore, the correct narrative of the case is that the **main path becomes Samba**.

Reasonable inference

Although the exact exploit syntax is not preserved, the link between Samba 3.0.20 and CVE-2007-2447 is strong enough to justify the technical decision to move the focus to that line. This does not invent steps: it only explains the reasoning that fits the preserved evidence.

Reusable lesson

When one promising path loses strength and another appears more solid, the change of focus should be written as a **reasoned technical decision**, not as an abrupt jump. That kind of pivot is exactly what is worth studying and documenting.

Exploitation validated through CVE-2007-2447

Verified fact

The central data point of this phase is very clear:

- after exploiting CVE-2007-2447, the obtained shell corresponds to root.

What is demonstrated

Although the full exploitation command is not preserved, three fundamental things are demonstrated:

1. the effective exploitation line was **Samba / CVE-2007-2447**;
2. the useful access does not end in a limited user;
3. the execution context becomes **directly privileged**.

What is not preserved

This specific detail must be marked as **pending verification**:

- the **exact command** or **complete syntax** used to launch the Samba exploitation in the historical resolution.

This does not invalidate the case chain. It simply requires documenting it honestly:

- **validated result**;
- **exact execution detail not preserved**.

Why this matters methodologically

On easy machines, it is common to fall into a bad documentation habit: filling gaps with memory, habit, or the "typical version" of the exploit. That should not be done here. The value of the case is not pretending to have precision that does not exist, but teaching how to distinguish between:

- what is **really validated**;
- what is a **reasonable reading**;
- what remains **pending verification**.

What changes afterward

Because the valid shell is already root, the case narrative changes a lot compared with longer machines. There is no classic phase of:

- obtaining a low-privileged user;
- extensive local enumeration;
- separate later privilege escalation.

Here the useful exploitation already provides privileged context. Therefore, the following phases should not be dressed up as if there had been a long escalation that did not really exist.

Reusable lesson

An exploitation that ends directly in `root` forces the way the case is narrated to be adjusted. If a fixed structure is kept without thinking, there is a risk of inventing an "escalation" that was not actually a separate phase.

Obtaining user.txt

What was executed

The user flag read is documented with this path and these commands:

```
cd /home/makis  
ls -la  
cat user.txt
```

Why this step is done this way

Once privileged context has been obtained, reading `user.txt` is no longer an access maneuver but a **validation of control over the system**. The objective here is not to escalate, but to confirm that the reached context allows browsing the system and accessing the relevant paths.

What each command does

- `cd /home/makis` places the session in the user's directory;
- `ls -la` shows files, permissions, and hidden content;
- `cat user.txt` displays the flag.

What was actually observed

The verified flag was:

- `60fc5d64febbdebfe8cc331838bff0b0`

Precision note

Although the document preserves this part as "obtaining `user.txt`", it should be explained clearly:

- it does not represent an independent low-privilege phase;
- it is not evidence of a limited user shell;
- it is a read performed **after** the valid exploitation had already provided `root` context.

What this phase teaches

In a mechanical writeup, it would be easy to write "user" and make it seem as if a first half of the case ended there. Here that would be misleading. The correct reading is that user . txt is part of **privileged post-access**, not a separate user acquisition phase.

Obtaining root.txt

What was executed

The final flag read is documented with:

```
cd /root
ls -la
cat root.txt
```

Why this step has documentary value

Although the context is already privileged, it is useful to document this phase explicitly for two reasons:

1. it formally closes the resolution of the case;
2. it confirms that the privileged access was not apparent or partial, but sufficient to reach the classic administration path.

What was actually observed

The verified flag was:

- c80b43503b56dc7b0dc82643157b4329

How it should be interpreted

With the reading of root . txt, the documented chain is closed. From this point on, there is no additional escalation to describe, only complementary observations about the behavior of some services and the methodological value of the case.

Complementary technical observations

This phase is not part of the main access path, but it preserves useful signals that deserve to be explained.

11.1 Listening services and real exposure

The historical evidence preserves this check:

```
netstat -tnlp
```

Annotated result:

- cause of the lack of external accessibility to certain locally visible ports: `firewall`.

What this means

The observation helps resolve a typical lab doubt: a port can appear to be listening locally and yet not be effectively available from outside. That difference between **local listening** and **real exposure** is important when interpreting apparently contradictory results.

11.2 Port associated with the VSFTPD backdoor

The material also notes that the port associated with the backdoor is:

- 6200

And preserves an additional check:

```
ss -tnlp | grep 6200
```

Annotated result:

- Yes, it listens.

How to read this block without mixing it with the main path

Here it is worth being especially careful. The available evidence leaves two simultaneous signals:

- the main Metasploit test against VSFTPD **did not produce a useful session**;
- the material also notes that port 6200 **did listen**.

The most prudent reading is not to force one of the two and erase the other, but to integrate them correctly:

- the expected backdoor behavior **seems to have been observed** partially or transiently;
- but that observation **did not become** the documented useful resolution path;
- therefore, the case should still be narrated as a machine solved through **Samba**, not through FTP.

Reusable lesson

Not every interesting signal should take the center of the writeup. Some belong better in a complementary observations section, where they can be preserved without deforming the main chain of the case.

Final technical summary of the real chain

Reconstructed technical chain

1. Initial enumeration of the top 1000 TCP ports.
2. Detection of open ports 21, 22, 139, 445.
3. Identification of vsFTPD 2.3.4 on FTP.
4. Test of the vsftpd_234_backdoor path without a documented useful session.
5. Identification of Samba 3.0.20 on 139/445.
6. Technical association with CVE-2007-2447.
7. Validated exploitation with resulting shell as root.
8. Reading of user.txt in /home/makis/user.txt.
9. Reading of root.txt in /root/root.txt.
10. Complementary checks about firewall and the behavior of port 6200.

Overall didactic reading

The machine leaves a very clean and very useful chain for study:

short enumeration → initially promising famous path → non-productive test → pivot to legacy service → exploitation with direct root → flag reading

That pattern is valuable because it does not depend on a long chain or artificial complexity. Above all, it teaches how to **read the evidence well and avoid automatic narration**.

Reusable lessons for PENTEST-STUDIO

What technical pattern this machine teaches

- Brief initial enumeration sufficient to prioritize services.
- Version confirmation before talking about exploitation.
- Explicit discard of an apparent but non-productive path.
- Shift of focus to a more solid legacy service.
- Exploitation that immediately provides root context.

What it corrects compared with common misreadings

- Not every apparently vulnerable version should be assumed as the final path simply because of its fame.
- **Tested line** and **actually exploited line** must be separated.
- If the initial shell is already root, a non-existent later escalation should not be narrated.
- When an exact piece of the exploit is missing, it should be marked as **pending verification** and not filled in from memory or assumption.

What it contributes to the phase 1 checklist

- always note the **exact service version** before jumping to CVEs;
- clearly record when a path is **discarded with evidence**;
- fix the **shell context** immediately after exploitation;
- separate access acquisition from flag reading.

What it contributes to the master roadmap

- reinforces the branch of **legacy services** and historically exploitable versions;
- forces the inclusion of a **validation and discard** step for famous paths before changing service;
- reminds us that old SMB can offer a stronger path than FTP even when both appear promising at first.

Applicable sub-roadmaps

- **Applicable:** SMB/Samba services branch and legacy software exploitation.
- **Not applicable:** web-base and web-auth/panel, because the case does not follow a web chain.